Multiple-Choice Questions on Sorting and Searching

1. The decision to choose a particular sorting algorithm should be made based on which of the following?

   I. Run-time efficiency of the sort
   II. Size of the array
   III. Space efficiency of the algorithm

   (A) I only
   (B) III only
   (C) I and II only
   (D) I, II, and III

2. The following code fragment does a sequential search to determine whether a given integer, `value`, is stored in an array `a[0] ... a[n-1]`.

```
int i = 0;
while (/* boolean expression */)
{
   i++;
}
if (i == n)
   return -1; //value not found
else
   return i;  //value found at
              //location i
```

   Which of the following should replace `/* boolean expression */` so that the algorithm works as intended?

   (A) `value != a[i]`
   (B) `i < n && value == a[i]`
   (C) `value != a[i] && i < n`
   (D) `i < n && value != a[i]`

3. A feature of data that is used for a binary search but not necessarily used for a sequential search is

   (A) length of list.
   (B) order of data.
   (C) smallest value in the list.
   (D) median value of the data.

4. Array `unsortedArr` contains an unsorted list of integers. Array `sortedArr` contains a list of integers sorted in increasing order. Which of the following operations is more efficient for `sortedArr` than `unsortedArr`? Assume the most efficient algorithms are used.

   I. Inserting a new element
   II. Searching for a given element
   III. Computing the mean of the elements

   (A) I only
   (B) II only
   (C) III only
   (D) I and II only

5. An algorithm for searching a large sorted array for a specific value x compares every third item in the array to x until it finds one that is greater than or equal to x. When a larger value is found, the algorithm compares x to the previous two items. If the array is sorted in increasing order, which of the following describes all cases when this algorithm uses fewer comparisons to find x than would a binary search?

   (A) It will never use fewer comparisons.
   (B) When x is in the middle position of the array
   (C) When x is very close to the beginning of the array
   (D) When x is very close to the end of the array

6. Assume that `a[0] ... a[N-1]` is an array of N positive integers and that the following assertion is true.

   `a[0] > a[k]` for all k such that $0 < k < N$

   Which of the following *must* be true?

   (A) The array is sorted in ascending order.
   (B) The array is sorted in descending order.
   (C) `a[0]` holds the smallest value in the array.
   (D) `a[0]` holds the largest value in the array.

7. The following code is designed to set `index` to the location of the first occurrence of `key` in array `a` and to set `index` to –1 if `key` is not in `a`.

```
index = 0;
while (a[index] != key)
    index++;
if (a[index] != key)
    index = -1;
```

In which case will this program *definitely* fail to perform the task described?

(A) When `key` is the first element of the array
(B) When `key` is the last element of the array
(C) When `key` is not in the array
(D) When `key` equals `a[key]`

8. Refer to method `search`.

```
/** Returns value k such that -1 <= k <= v.length-1.
 *  If k >= 0 then v[k] == key.
 *  If k == -1, then key != any of the elements in v.
 */
public static int search(int[] v, int key)
{
    int index = 0;
    while (index < v.length && v[index] < key)
        index++;
    if (v[index] == key)
        return index;
    else
        return -1;
}
```

Assuming that the method works as intended, which of the following should be added to the precondition of `search`?

(A) `v` is sorted smallest to largest.
(B) `v` is unsorted.
(C) There is at least one occurrence of `key` in `v`.
(D) `key` occurs no more than once in `v`.

9. Consider the following class.

```
/** A class that sorts an array of Integer objects from
 * largest to smallest using a selection sort.
 */
public class Sorter
{
    private Integer[] a;

    public Sorter(Integer[] arr)
    { a = arr; }

    /** Swap a[i] and a[j] in array a. */
    private void swap(int i, int j)
    { /* implementation not shown */ }

    /** Sort array a from largest to smallest using selection sort.
     * Precondition: a is an array of Integer objects.
     */
    public void selectionSort()
    {
        for (int i = 0; i < a.length - 1; i++)
        {
            //find max element in a[i+1] to a[n-1]
            Integer max = a[i];
            int maxPos = i;
            for (int j = i + 1; j < a.length; j++)
                if (max.compareTo(a[j]) < 0) //max less than a[j]
                {
                    max = a[j];
                    maxPos = j;
                }
            swap(i, maxPos);    //swap a[i] and a[maxPos]
        }
    }
}
```

If an array of `Integer` contains the following elements, what would the array look like after the third pass of `selectionSort`, sorting from high to low?

89 42 –3 13 109 70 2

(A) 109 89 70 13 42 –3 2
(B) 109 89 70 42 13 2 –3
(C) 109 89 70 –3 2 13 42
(D) 109 89 42 –3 13 70 2

Questions 10–14 are based on the `binSearch` method and the private instance variable a for some class.

```
private int[] a;

/** Does binary search for key
 *  in array
 *  a[0]...a[a.length-1], sorted in
 *  ascending order.
 *  Returns index such that
 *  a[index]==key.
 *  If key is not in a, returns -1.
 */
public int binSearch(int key)
{
  int low = 0;
  int high = a.length - 1;
  while (low <= high)
  {
     int mid = (low + high) / 2;
     if (a[mid] == key)
        return mid;
     else if (a[mid] < key)
        low = mid + 1;
     else
        high = mid - 1;
  }
  return -1;
}
```

A binary search will be performed on the following list.

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]
4	7	9	11	20	24	30	41

10. To find the key value 27, the search interval *after* the first pass through the `while` loop will be

(A) a[0] ...a[7]
(B) a[5] ...a[6]
(C) a[4] ...a[7]
(D) a[2] ...a[6]

11. How many iterations will be required to determine that 27 is not in the list?

(A) 1
(B) 3
(C) 4
(D) 8

12. What will be stored in y after executing the following?

```
int y = binSearch(4);
```

(A) 20
(B) 4
(C) 0
(D) -1

13. If the test for the `while` loop is changed to

```
while (low < high)
```

the `binSearch` method does not work as intended. Which value in the given list will not be found?

(A) 4
(B) 7
(C) 11
(D) 30

14. For `binSearch`, which of the following assertions will be true following every iteration of the `while` loop?

(A) a[low] ≤ key ≤ a[high]
(B) low ≤ mid ≤ high
(C) key = a[mid], or a[low] ≤ key ≤ a[high]
(D) key = a[mid], or a[low] ≤ key ≤ a[high], or key is not in array a.

15. A large sorted array containing about 30,000 elements is to be searched for a value `key` using an iterative binary search algorithm. Assuming that `key` is in the array, which of the following is closest to the smallest number of iterations that will guarantee that `key` is found? Note: $10^3 \approx 2^{10}$.

(A) 15
(B) 30
(C) 100
(D) 3000

For Questions 16–19 refer to the `insertionSort` method and the private instance variable `a`, both in a `Sorter` class.

```
private Integer[] a;

/** Precondition: a[0],a[1]...
 *        a[a.length-1] is
 *        an unsorted array of
 *        Integer objects.
 *  Postcondition: Array a is
 *        sorted in
 *        descending order.
 */
public void insertionSort()
{
    for (int i = 1;
    i < a.length; i++)
    {
        Integer temp = a[i];
        int j = i - 1;
        while (j >= 0 &&
        temp > a[j])
        //temp and a[j]
        //are unboxed
        {
            a[j+1] = a[j];
            j--;
        }
        a[j+1] = temp;
    }
}
```

16. An array of `Integer` is to be sorted biggest to smallest using the `insertionSort` method. If the array originally contains

    1 7 9 5 4 12

    what will it look like after the third pass of the `for` loop?

    (A) 9 7 1 5 4 12
    (B) 9 7 5 1 4 12
    (C) 12 9 7 1 5 4
    (D) 12 9 7 5 4 1

17. When sorted biggest to smallest with `insertionSort`, which list will need the fewest changes of position for individual elements?

    (A) 5, 1, 2, 3, 4, 9
    (B) 9, 5, 1, 4, 3, 2
    (C) 9, 4, 2, 5, 1, 3
    (D) 3, 2, 1, 9, 5, 4

18. When sorted biggest to smallest with `insertionSort`, which list will need the greatest number of changes in position?

    (A) 5, 1, 2, 3, 4, 7, 6, 9
    (B) 9, 4, 6, 2, 1, 5, 1, 3
    (C) 9, 6, 9, 5, 6, 7, 2, 0
    (D) 3, 2, 1, 0, 9, 6, 5, 4

19. While typing the `insertionSort` method, a programmer by mistake enters

    ```
    while (temp > a[j])
    ```

    instead of

    ```
    while (j >= 0 && temp > a[j])
    ```

    Despite this mistake, the method works as intended the first time the programmer enters an array to be sorted in descending order. Which of the following could explain this?

    I. The first element in the array was the largest element in the array.
    II. The array was already sorted in descending order.
    III. The first element was less than or equal to all the other elements in the array.

    (A) I only
    (B) III only
    (C) I and II only
    (D) II and III only

20. The elements in a long list of integers are roughly sorted in decreasing order. No more than 5 percent of the elements are out of order. Which of the following is a valid reason for using an insertion sort rather than a selection sort to sort this list into decreasing order?

I. There will be fewer comparisons of elements for insertion sort.
II. There will be fewer changes of position of elements for insertion sort.
III. There will be less space required for insertion sort.

(A) I only
(B) II only
(C) I and II only
(D) I, II, and III

21. A large array of lowercase characters is to be searched for the pattern "pqrs." The first step in a very efficient searching algorithm is to look at characters with index

(A) 0, 1, 2, ... until a "p" is encountered.
(B) 0, 1, 2, ... until any letter in "p" ... "s" is encountered.
(C) 3, 7, 11, ... until an "s" is encountered.
(D) 3, 7, 11, ... until any letter in "p" ... "s" is encountered.

22. The array `names[0]`, `names[1]`, ..., `names[9999]` is a list of 10,000 name strings. The list is to be searched to determine the location of some name `X` in the list. Which of the following preconditions is necessary for a binary search?

(A) There are no duplicate names in the list.
(B) The number of names *N* in the list is large.
(C) The list is in alphabetical order.
(D) Name `X` is definitely in the list.

23. A binary search is to be performed on an array with 600 elements. In the *worst* case, which of the following best approximates the number of iterations of the algorithm?

(A) 6
(B) 10
(C) 100
(D) 600

24. A worst-case situation for insertion sort would be

I. A list in correct sorted order.
II. A list sorted in reverse order.
III. A list in random order.

(A) I only
(B) II only
(C) I and II only
(D) II and III only

25. Consider a binary search algorithm to search an ordered list of numbers. Which of the following choices is closest to the maximum number of times that such an algorithm will execute its main comparison loop when searching a list of 1 million numbers?

(A) 6
(B) 20
(C) 120
(D) 1000

26. Consider these three tasks.

I. A sequential search of an array of *n* names
II. A binary search of an array of *n* names in alphabetical order
III. An insertion sort into alphabetical order of an array of *n* names that are initially in random order

For large *n*, which of the following lists these tasks in order (from least to greatest) of their average case run times?

(A) II I III
(B) I II III
(C) II III I
(D) III II I

Questions 27–29 refer to the Hi-Lo game described below.

Consider the problem of writing a Hi-Lo game in which a user thinks of an integer from 1 to 100 inclusive and the computer tries to guess that number. Each time the computer makes a guess, the user makes one of three responses.

- "Lower" (i.e., the number is lower than the computer's guess)
- "Higher" (i.e., the number is higher than the computer's guess)
- "You got it in < ***however many*** > tries!"

27. Suppose the game is programmed so that the computer uses a binary search strategy for making its guesses. What is the maximum number of guesses the computer could make before guessing the user's number?

(A) 25
(B) 10
(C) 7
(D) 6

28. Suppose the computer used a *sequential search* strategy for guessing the user's number. What is the maximum number of guesses the computer could make before guessing the user's number?

(A) 100
(B) 99
(C) 50
(D) 25

29. Using a sequential search strategy, how many guesses *on average* would the computer need to guess the number?

(A) Between 51 and 99
(B) 50
(C) 25
(D) Fewer than 25